RESULTS

HelixLLM Phase-3 CPU Translation — NLLB-200-CTranslate2 PRIMARY LANE — END-TO-END PROOF (§11.4.108)

Status	**COMPLETE — ALL-GREEN on the design-default PRIMARY lane.** No substitution needed in the final run (an earlier attempt hit a transient host-load fault, root-caused and fixed — see §3).
Run-id	`phase3_translation_nllb_20260707`
Date	2026-07-07T11:38:04Z → 11:38:25Z (UTC), host `x86_64`, podman 5.7.1, rootless
Branch / Track	`feature/helixllm-full-extension` · `(T1/feature/helixllm-full-extension)`
Design	`docs/research/07.2026/00_master/TRANSLATION_PROVIDER.md` §1.2/§3/§4
Extends	`docs/qa/phase3_translation_20260707/` — the LibreTranslate **FALLBACK** lane, already shipped + proven. This run proves the design's **PRIMARY** lane: NLLB-200-distilled-600M via **CTranslate2**.
Pattern mirrored	`docs/qa/phase3_embeddings_20260706/harness/` (embeddings proof)
Lane PROVEN here	**NLLB-200-distilled-600M via CTranslate2 (CPU)**, behind a thin bespoke HTTP shim (`harness/shim/{Dockerfile,server.py}`) — the design's documented default (`TRANSLATION_PROVIDER.md` §1.2)

1. §11.4.150 deep multi-angle research (before implementation)

Angles covered (dated 2026-07-07, this session):

1. **CTranslate2 + NLLB-200 CPU serving / community precedent** — confirmed active usage (`entai2965/nllb-200-distilled-600M-ctranslate2`, `JustFrederik/nllb-200-distilled-600M-ct2-int8`, `winstxnhdw/nllb-api`, `any35/ctranslate2-server`) and the canonical conversion command `ct2-transformers-converter --model facebook/nllb-200-distilled-600M --output_dir <dir>` (requires `transformers>=4.21.0`).

2. **CTranslate2 + transformers tokenizer flow** (the exact code path this shim uses) — confirmed from **two independent sources** (§11.4.150(B)): the OpenNMT canonical docs (`opennmt.net/CTranslate2/guides/transformers.html`) AND the `entai2965` model card, cross-checked and found identical:

   ```
   translator = ctranslate2.Translator(model_dir, device="cpu")
   tokenizer  =
           transformers.AutoTokenizer.from_pretrained(model_dir,
           src_lang=src_lang)
   source     =
           tokenizer.convert_ids_to_tokens(tokenizer.encode(text))
   results    = translator.translate_batch([source],
           target_prefix=[[tgt_lang]])
   target     = results[0].hypotheses[0][1:]           # drop
           the leading lang token
   out_text   =
           tokenizer.decode(tokenizer.convert_tokens_to_ids(target))
   ```

   This is exactly the code path implemented in `harness/shim/server.py::translate_one()`.

3. **FLORES-200 language-code format** — confirmed `eng_Latn` / `deu_Latn` / `fra_Latn` (ISO-639-3 + ISO-15924 script tag), the format NLLB's tokenizer requires for `src_lang` / `target_prefix`.

4. **Pre-converted-model file-listing verification** (avoids an in-container HF→CT2 conversion step, which needs `torch` + roughly doubles the image/RAM footprint) — fetched `entai2965/nllb-200-distilled-600M-ctranslate2`'s file tree directly BEFORE choosing it: `model.bin` (2.46 GB), `sentencepiece.bpe.model`, `shared_vocabulary.json`, `tokenizer.json`, `tokenizer_config.json`, `special_tokens_map.json`, `config.json`,

`LICENSE.model.md` — a fully self-contained CT2+tokenizer repo (no missing-file boot risk). This verification is exactly why the primary lane booted cleanly once the thread-limit issue (§3) was fixed — no missing-tokenizer surprises.

5. **License note** (re-verified from the design doc) — NLLB weights are **CC-BY-NC-4.0 (non-commercial)**; flagged, not resolved, here (design doc Open Question Q5 — unchanged by this proof).

Sources verified (§11.4.99/§11.4.150, accessed 2026-07-07)

- https://huggingface.co/entai2965/nllb-200-distilled-600M-ctranslate2
- https://huggingface.co/entai2965/nllb-200-distilled-600M-ctranslate2/tree/main
- https://opennmt.net/CTranslate2/guides/transformers.html
- https://github.com/OpenNMT/CTranslate2/blob/master/docs/guides/transformers.md
- https://huggingface.co/JustFrederik/nllb-200-distilled-600M-ct2-int8 (documented fallback candidate, not needed in the final run)
- https://huggingface.co/OpenNMT/nllb-200-distilled-1.3B-ct2-int8
- https://forum.opennmt.net/t/nllb-200-with-ctranslate2/5090
- https://github.com/winstxnhdw/nllb-api
- (carried from the design spike, re-affirmed) https://github.com/argosopentech/argos-translate ; https://docs.libretranslate.com/api/operations/translate/

---

2. What was built

- **`harness/shim/{Dockerfile,server.py}`** — a minimal `python:3.11-slim` image (`ctranslate2`, `transformers`, `sentencepiece`, `protobuf`, `huggingface_hub` — **no torch**, since only the tokenizer, not a conversion, is needed at runtime) exposing `GET /health` and `POST /translate {"q","source","target"} -> {"translatedText"}`. Model load runs in a background thread so `/health` can report `503 loading` immediately, `200 ok` once ready, `500` with the captured error on load failure (never a silent hang).
- **`harness/compose.phase3translatenllb.yml`** — booted via the **containers submodule** `compose.Orchestrator` (§11.4.76), rootless podman (§11.4.161), **NO GPU** device anywhere in the spec. Host port **18436** (coder `:18434` and the embeddings tier's `:18435` untouched). Persistent external HF-model cache volume `helixllm-nllb-cache` (§11.4.77).

- **`harness/main.go`** + **`run_proof.sh`** — the Go proof harness (boot/probe/analyze/determinism/selfvalidate subcommands) + the orchestrating shell script, mirroring the proven embeddings/LibreTranslate harness shape.

---

3. Root-cause investigation + fixes (§11.4.102/ §11.4.146 — reproduce-first, then fix)

The **first** attempt at this proof did **not** reach ALL-GREEN on the first try. Per the Iron Law (§11.4.102: no fixes without root-cause investigation first), each defect was root-caused from captured evidence before any fix — none of this was guessed (§11.4.6). All investigation evidence is preserved under `investigation_first_attempt/`.

1. **PRIMARY lane container crashed during model load** — `investigation_first_attempt/22_shimlogs_primary.txt` captured the exact traceback: `OpenBLAS blas_thread_init: pthread_create failed for thread 52 of 64: Resource temporarily unavailable ... RLIMIT_NPROC 4096 current, 5120 max`. **Root cause**: OpenBLAS (used internally by `ctranslate2`) tries to spin up one thread **per detected host CPU** (64 on this host) at `ctranslate2.Translator()` construction time — independent of the `intra_threads` parameter passed to CT2 itself — and this exhausted the container's process/thread ulimit under real, concurrent host load (see point 3). **Fix**: `shim/server.py` now sets `OPENBLAS_NUM_THREADS` / `OMP_NUM_THREADS` / `MKL_NUM_THREADS` explicitly (env-injected via the compose file, `NLLB_BLAS_THREADS`, default 4) **before** any BLAS-using library is imported.
2. **A "fallback" run reported healthy in ~8 seconds — implausible for a 600 MB+ model — investigated and found to be a real bug**: the shim's original `load_model()` checked only `os.path.exists(MODEL_DIR/model.bin)` to decide whether to (re)download, sharing **one** cache directory across every `MODEL_REPO` lane on the same persistent volume. When the PRIMARY lane's download completed (2.46 GB, confirmed present on-disk by direct inspection of the podman volume mountpoint) but the process then crashed (point 1) **after** the download, a **subsequent** container booted with a **different** `MODEL_REPO` (the fallback) found `model.bin` already present and silently served the **wrong** (primary lane's) model under the fallback's name — a stale-shadow bluff (§11.4.108/§11.4.139). **Fix**: the shim now keys its actual working directory by the repo id (`MODEL_DIR/<repo>.replace("/","__")`), making cross-lane cache collision structurally impossible; `run_proof.sh` additionally cross-checks the `/health` response's reported `model` field against the requested repo before

accepting a lane as served (`21b_served_model_primary.txt` captures this check passing in the final run).

3. **Host-level `rootlessport fork/exec: resource temporarily unavailable` at `podman-compose up` time itself** (container-runtime level, not fixable from inside the container) — investigated via `/proc/loadavg` (`3/5012` total scheduled entities) and `ulimit -u` (4096 for this user), plus direct observation of concurrent sibling work-stream tracks on the same host/repo (Tesseract-OCR and Whisper-STT Phase-3 proofs, per §11.4.174/§11.4.176 — confirmed via a co-running, unrelated `git diff` process enumerating those tracks' paths). This is a genuine, transient, host-load fault — not a model/CT2 incompatibility, since it fails at container-start, before any model code runs. **Fix**: `run_proof.sh`'s `run_lane` now retries `run_lane_once` up to 4 times with linear backoff (10s·k), and `poll_health` fast-fails after 10 consecutive polls stuck in container state `created` (≈40s) instead of burning the full timeout on a container that will never start.
4. **Two bash bugs in `run_proof.sh` itself, caught because the buggy first run produced an impossible verdict** ("ALL-GREEN" with no green-record files on disk): (a) `PAIRS=(en->de en->fr)` — unquoted, the literal `>` is parsed by bash as an output-redirection operator **anywhere** it appears in an unquoted word, which was a silent syntax error that skipped the entire probe loop; (b) `{ block; } | tee file` runs the **left side of a pipe in a subshell** in bash — verified empirically (`X=0` assigned inside such a block was lost in the parent shell after the pipe) — so `GREEN_ALL=0` assigned inside such a block on a genuine failure was silently lost, and the script printed "ALL-GREEN" anyway. **Fix**: array elements quoted; the state-carrying blocks now redirect straight to a file (no pipe, no subshell) and are `cat`'d afterward for the same console visibility.

None of these four issues were a genuine NLLB-200/CTranslate2 **incompatibility** — all four were fixable at the shim/harness layer. After the fixes, the **PRIMARY lane** (`entai2965/nllb-200-distilled-600M-ctranslate2`) booted and passed cleanly on the **first attempt** of the corrected run (`health OK after 2 polls` — fast because the model was already fully downloaded from the earlier attempt and reused via the persistent cache, §11.4.77).

---

4. Verdict — RUNTIME SIGNATURE (honest, §11.4.6)

CAPABILITY PROVEN — PRIMARY lane, no substitution.

```
[RUNTIME-SIGNATURE(en->de)] PASS notIdentity=true allKeywordsOK=true anyKe
[DETERMINISM]               PASS forward byte-identical across two identic
[RUNTIME-SIGNATURE(en->fr)] PASS notIdentity=true allKeywordsOK=true anyKe
[DETERMINISM]               PASS forward byte-identical across two identic
[SELF-VALIDATION]           PASS: analyzer PASSes golden-good and FAILs al
```

```
served-model cross-check:   requested=entai2965/nllb-200-distilled-600M-ct
                             reported=entai2965/nllb-200-distilled-600M-ct
ALL-GREEN: runtime signature + determinism + self-validation PASS (lane=en
```

Captured real translations (`green_record_*.json`)

Pair	source	forward (real NLLB-CT2 output)	required keywords	verdict
en→de	"The house is blue."	**"Das Haus ist blau."**	ALL of `haus` ✓; ANY of `blau/blaues/blaue` ✓ (`blau` present)	**PASS**
en→fr	"The cat sleeps."	**"Le chat dort."**	ALL of `chat` ✓; ANY of `dort/dormir` ✓ (`dort` present)	**PASS**

Both forward outputs are genuinely different from their source strings (not-identity ✓) and are real, fluent, grammatically-correct translations — not a copy, not garbage, not the wrong language.

Determinism (§11.4.50)

Two identical `POST /translate` requests per pair produced **byte-identical** `translatedText`: - en→de: `"Das Haus ist blau."` == `"Das Haus ist blau."` (`green_record_en_de_1.json` / `_2.json`) - en→fr: `"Le chat dort."` == `"Le chat dort."` (`green_record_en_fr_1.json` / `_2.json`)

(CTranslate2 `beam_size=1`, i.e. greedy decoding — deterministic by construction, no sampling.)

---

5. Analyzer is non-bluff (§11.4.107(10) / §11.4.115)

`10_red_baseline.txt` — the exact untranslated-passthrough "warming" bluff (an echo-stub that returns q unchanged) fed to the analyzer for **both** pairs, **before** the real lane was asked to PASS:

```
[RUNTIME-SIGNATURE(en->de)] FAIL notIdentity=false allKeywordsOK=false any
    reason: identity/passthrough: forward "The house is blue." == source "
    reason: missing required keyword(s) [haus] in forward "The house is bl
    reason: none of the alternative keyword(s) [blau blaues blaue] present
[RUNTIME-SIGNATURE(en->fr)] FAIL notIdentity=false allKeywordsOK=false any
    reason: identity/passthrough: forward "The cat sleeps." == source "The
```

```
    reason: missing required keyword(s) [chat] in forward "The cat sleeps.
    reason: none of the alternative keyword(s) [dort dormir] present in fo
RED-OK: echo-stub correctly FAILED the runtime signature for both pairs
```

`12_self_validation.txt` — golden-good = the REAL captured en→de record; every golden-bad variant correctly FAILS:

Fixture	expect	notIdentity	allKeywordsOK	anyKeywordOK	result
GOLDEN-GOOD (real “Das Haus ist blau.”)	PASS	true	true	true	**PASS** ✓
BAD identity/ echo (forward == source)	FAIL	false	false	false	**FAIL** ✓
BAD empty (forward == ““)	FAIL	false	false	false	**FAIL** ✓
BAD wrong-language (real en→fr output “Le chat dort.” substituted in)	FAIL	true	false	false	**FAIL** ✓

The wrong-language fixture is deliberately a **genuine, fluent, correct** translation — just for the *other* request — proving the analyzer rejects “a real translation, just not the one asked for,” not merely garbage (mirrors the CX-05 anti-gaming construction used by the sibling LibreTranslate proof).

---

6. Containerization (§11.4.76 / §11.4.161, NO GPU)

- Booted via `digital.vasic.containers/pkg/compose.NewDefaultOrchestrator` (`harness/main.go`), **not** ad-hoc podman/docker — rootless podman.

- `compose.phase3translatenllb.yml` carries **no literal** host/port/model/thread value (§CONST-045/046) — all injected by `run_proof.sh` env vars.
- **NO GPU**: no `--device nvidia.com/gpu`, no GPU flag anywhere; the shim always constructs `ctranslate2.Translator(..., device="cpu")`.
- Image `localhost/helixllm-nllb-shim:latest`, built via `podman build` from `harness/shim/` (the same auxiliary-podman-command precedent as the prior harnesses' `podman pull`/`podman volume create` steps — the actual service UP/DOWN lifecycle still goes exclusively through the containers-submodule orchestrator).
- Host port **18436** (coder `:18434`, embeddings `:18435` untouched and verified free pre-boot, `00_preflight.txt`).
- Persistent external cache volume `helixllm-nllb-cache` (§11.4.77 re-obtainable — re-downloads from `MODEL_REPO` if dropped), now keyed per-repo internally (§3, point 2).

Single-owner teardown + coder untouched (§11.4.119)

`29_teardown.txt` / `29b_post_teardown.txt`:

```
DOWN-OK: phase3translatenllb_primary nllb-shim (volumes removed) via conta
nllb-shim containers (expect none):
  (none — removed)
coder still running (untouched):
helixllm-coder Up 22 hours
```

`24_container_state.txt` confirms both containers coexisted correctly during the run (`helixllm-coder ... Up 22 hours` + `phase3translatenllb_primary_nllb-shim_1 ... Up 4 seconds`).

---

7. Four-layer verification map (§11.4.108)

1. **SOURCE** — `harness/{main.go,shim/server.py,compose.phase3translatenllb.yml,run_proof.sh}` committed.
2. **ARTIFACT** — shim image built (`01_image_build.txt`); model weights present at the repo-keyed cache path (verified by direct filesystem inspection, 2.46 GB `model.bin` + tokenizer files); `/health` green.
3. **RUNTIME-ON-CLEAN-TARGET** — unique per-run project name + pre-clean `boot-down` before boot ⇒ a genuinely fresh container; the §4 (task-spec) runtime signature verified against it, including the served-model cross-check (`21b_served_model_primary.txt`) proving the *running* artifact matches the *requested* one. **This is the definition of done.**
4. **USER-VISIBLE** — a real client `POST /translate` for "The house is blue." returns "Das Haus ist blau." (and "The cat sleeps." → "Le chat dort.") —

correct, fluent, right-language translations a downstream doc/glossary flow could consume.

---

8. Honest boundary (§11.4.6)

- This proves the design's **PRIMARY** lane (NLLB-200-distilled-600M via CTranslate2, CPU) for **two** language pairs (en→de, en→fr) via a **proof harness shim** — it does **not** implement the full HelixLLM gateway `/v1/translate` route, auto-detect, `/languages`, HTML-format DNT protection, or glossary/formality control (design doc §2/§5 — future work, same non-scope as the already-shipped LibreTranslate fallback proof).
- **License**: NLLB weights are CC-BY-NC-4.0 (non-commercial) — unresolved, flagged per design doc Open Question Q5.
- **Model file provenance**: `entai2965/nllb-200-distilled-600M-ctranslate2` is a **community** CT2 conversion of `facebook/nllb-200-distilled-600M`, not an official OpenNMT-published int8 checkpoint for the 600M size (OpenNMT publishes official int8 checkpoints for the 1.3B/3.3B sizes only, per research §1). Quantization of this specific repo's `model.bin` was not explicitly confirmed by its model card (file size ~2.46 GB is consistent with float32, not int8) — this proof establishes that the **PRIMARY NLLB-200-distilled-600M-via-CTranslate2 lane genuinely works end-to-end**; it does not additionally establish int8 quantization was in effect. A follow-up could pin an int8-confirmed conversion (e.g. `JustFrederik/nllb-200-distilled-600M-ct2-int8`, held in reserve as the documented fallback and NOT exercised in this final run) for a smaller memory/latency footprint.
- Two language pairs only (task spec); the design's full acceptance signature (§4 of `TRANSLATION_PROVIDER.md`) additionally calls for chrF-vs-golden + back-translation metamorphic scoring (as the sibling LibreTranslate proof implements) — this proof uses the task's specified keyword-substring + not-identity signature instead, which is a stricter, simpler, equally unfakeable check for exactly these two fixtures, but does not produce a continuous adequacy score. Both signatures are legitimate per-design; extending this harness with the chrF/back-translation triple is straightforward future work reusing the sibling harness's `chrF()` implementation.

---

9. Reproduce

```
cd docs/qa/phase3_translation_nllb_20260707/harness
./run_proof.sh
```

Boots the NLLB-CT2 shim via the containers submodule orchestrator, runs the RED baseline, probes both pairs twice (determinism), analyzes the runtime

signature, self-validates the analyzer, tears the container down (single-owner, `helixllm-coder` untouched), and regenerates every evidence file in this directory. `NLLB_HOST_PORT` / `NLLB_BLAS_THREADS` / `HEALTH_TIMEOUT` etc. are all env-tunable (see the script header). The `harness/phase3translatenllb.bin` build artifact and the persistent `helixllm-nllb-cache` podman volume are gitignored / external (§11.4.30 / §11.4.77) — the first run on a clean host downloads ~2.46 GB from Hugging Face into the cache; subsequent runs reuse it.

Investigation evidence for the root-cause narrative in §3 is preserved under `investigation_first_attempt/` (the OpenBLAS crash traceback, the stale-shadow fallback boot, and the resulting stale substitution note from the pre-fix run — kept for audit, not part of the final canonical proof sequence).

---

10. Composition footer — constitutional anchors touched

§11.4.6 (honest substitution — none was ultimately needed; honestly documents why an earlier attempt needed one) · §11.4.28/§11.4.35 (decoupled, project-supplies-its-own-harness) · §11.4.50 (determinism) · §11.4.74 (extend/reuse CTranslate2 + transformers, no bespoke NMT engine) · §11.4.76/§11.4.161 (containers submodule, rootless) · §11.4.77 (re-obtainable model cache) · §11.4.99/§11.4.150 (latest-source, multi-angle research, cited) · §11.4.102/§11.4.146 (reproduce-first, root-cause-before-fix, RED → fix → GREEN) · §11.4.107(10)/(15) (self-validated analyzer, RED polarity) · §11.4.108/§11.4.139 (four-layer verification on a clean, cross-checked artifact) · §11.4.119 (single-owner teardown, coder untouched) · §11.4.174 (shared-host process- ownership verification before diagnosing/acting) · §11.4.84 (no `git add -A`, scoped commit).